# Массивы

## Серия 3



## Как сдавать

Двенадцать задач по 12 баллов, всё на numpy. Теория — листок «Массивы в numpy». Массивы к задачам 1–6 лежат в `kontest/seriya-03/dannye/` и читаются так:

```
d = np.load("ocenki.npz")
o = d["o"]
```

В задачах 7–12 сдаётся файл с функцией: проверка сама вызывает её на своих массивах и сравнивает результат, дробные числа — с допуском.

```
python kontest.py 1a 3.5
python kontest.py 7 pairwise.py
python kontest.py itog
```

У задачи с ответом засчитывается последняя посылка, у функции — лучшая. Для функций проверке нужен numpy: запускай `kontest.py` тем питоном, где он стоит.

## Оси, маски, пропуски

Номера — с нуля, как в numpy. Ответ округляй, только если условие просит.

**1 (Журнал оценок).** `ocenki.npz`: массив `o` формы $(30, 8, 4)$ — ученик, предмет, четверть. Целые: от 2 до 5 — оценка, 0 — оценки нет. `predmety` — названия предметов по порядку.

а) Найди среднюю оценку по всему журналу, без нулей. Ответ округли до сотых.

б) По какому предмету средняя оценка, без нулей, самая низкая? Ответ — название.

в) Сколько учеников, у которых в четвёртой четверти по всем восьми предметам стоит 4 или 5?

**2 (Метеосеть).** `meteo.npz`: массив `t` формы $(12, 30, 24)$ — станция, день, час; значение — температура, `nan` — замера нет. `stancii` — названия станций по порядку.

а) На какой станции средняя температура за месяц, по всем её замерам, наибольшая? Ответ — название.

б) Суточная амплитуда станции — разность наибольшей и наименьшей температуры за день по её замерам; если в этот день у станции нет ни одного замера, амплитуды нет. Найди среднюю амплитуду по всем станциям и дням. Ответ округли до сотых.

в) Сколько часов — пар «день, час», — когда замер есть у всех двенадцати станций?

**3 (Снимок).** `snimok.npz`: массив `img` формы $(80, 120)$ и типа `uint8` — яркость пикселей от 0 до 255; первый номер — строка, второй — столбец.

а) Сколько пикселей ярче 200?

б) В каком столбце сумма яркостей наибольшая? Ответ — номер.

в) Сколько пар соседних по горизонтали пикселей, яркость которых различается больше чем на 50?

**4 (Продажи).** `prodazhi.npz`: `q` формы $(25, 40)$ — сколько штук каждого из 40 товаров продал каждый из 25 магазинов; `cena` — цена каждого товара в рублях; `region` — номер региона каждого магазина, от 0 до 4. Выручка магазина — сумма по товарам: штуки на цену.

а) Какой магазин получил наибольшую выручку? Ответ — номер.

б) Какова выручка всех магазинов региона 2? Ответ в рублях, с копейками.

в) Сколько товаров ни разу не продавались в магазинах региона 0?

**5 (Векторы слов).** `vektory.npz`: `v` формы $(300, 24)$ — векторы слов, `slova` — сами слова, коды вида `w042`, в том же порядке. Близость двух слов — косинус угла между их векторами.

а) Сколько векторов длиной больше 6?

б) Какое слово ближе всего к `w042`, не считая его самого?

в) Сколько пар слов $\{a, b\}$, $a \neq b$, у которых косинус больше 0,9?

**6 (Пропуски в ряду).** `ryad.npz`: `x` — 5000 замеров через равные промежутки, `nan` — пропуск. Первый и последний замеры есть. Участок пропусков — несколько пропусков подряд, слева и справа от которых стоят замеры.

а) Сколько пропусков?

б) Сколько участков пропусков?

в) Сколько пропусков в самом длинном участке?

г) Заполни каждый пропуск линейно между ближайшими замерами слева и справа, по номерам замеров. Найди среднее заполненного ряда. Ответ округли до тысячных.

## Без циклов

В каждой задаче сдаётся файл `.py` с одной функцией — имя и аргументы как в условии; печатать и читать ничего не нужно. Аргументы — массивы numpy и целые числа. Вернуть можно массив или список.

- Входные массивы менять нельзя: такая проверка не засчитывается.
- Полный балл — только без циклов: если в файле есть `for`, `while`, comprehension, `map` или `np.vectorize`, больше половины баллов не будет.
- Балл — доля пройденных проверок, их около десятка; примеры из условия баллов не дают. Ограничение — 2 секунды на проверку, самые большие массивы — до миллиона чисел.

**7 (Попарные расстояния).** `pairwise_l1(X, Y)`: `X` формы $(n, d)$, `Y` формы $(m, d)$. Верни матрицу $D$ формы $(n, m)$, где $D_{ij} = \sum_t |X_{it} - Y_{jt}|$ — манхэттенское расстояние между $i$-й строкой `X` и $j$-й строкой `Y`. $nmd \le 2 \cdot 10^6$.

```
pairwise_l1([[0, 0], [1, 2]], [[1, 1], [3, 0], [0, 2]])
  → [[2, 3, 2], [1, 4, 1]]

pairwise_l1([[0.5, -1, 2]], [[0, 0, 0], [1.5, -1, 2.5]])
  → [[3.5, 1.5]]
```

**8 (Среднее по группам).**
`group_mean(values, groups, k)`: `values` — $n$ чисел, `groups` — их группы, целые от 0 до $k-1$. Верни массив длины $k$: среднее `values` в каждой группе; у группы без единого элемента — `nan`. $n \le 10^6$, $k \le 10^5$.

```
group_mean([1, 2, 3, 4], [0, 1, 0, 1], 2)
  → [2.0, 3.0]

group_mean([5, 7, 1], [2, 2, 0], 4)
  → [1.0, nan, 6.0, nan]
```

**9 (Скользящий максимум).** `moving_max(a, w)`: `a` — $n$ чисел, $1 \le w \le n$. Верни массив длины $n - w + 1$, где $i$-й элемент — наибольшее из $a_i, \dots, a_{i+w-1}$. $n \le 10^6$.

```
moving_max([1, 3, 2, 5, 4], 2)
  → [3, 3, 5, 5]

moving_max([4, -1, 0.5, 0.5, 7, 2], 3)
  → [4.0, 0.5, 7.0, 7.0]
```

**10 (Самые длинные строки).** `top_k_rows(X, k)`: `X` формы $(n, d)$, $1 \le k \le n$. Верни номера $k$ строк с наибольшей евклидовой длиной, от самой длинной к короче; при равной длине раньше идёт строка с меньшим номером. $nd \le 2 \cdot 10^6$.

```
top_k_rows([[3, 4], [1, 0], [0, 6], [2, 2]], 2)
  → [2, 0]

top_k_rows([[1, 0], [0, 1], [1, 1], [0, 0]], 3)
  → [2, 0, 1]
```

**11 (Ранги).** `ranks(a)`: `a` — $n$ чисел. Ранг элемента — его место, если упорядочить числа по возрастанию, начиная с 1; у равных элементов ранг — среднее мест, которые они занимают вместе. Верни массив рангов в порядке элементов `a`. $n \le 10^6$.

```
ranks([30, 10, 20])
  → [3.0, 1.0, 2.0]

ranks([2, 1, 2, 3, 1])
  → [3.5, 1.5, 3.5, 5.0, 1.5]
```

**12 (Матрица ошибок).** `confusion(y_true, y_pred, k)`: истинные и предсказанные классы $n$ объектов, целые от 0 до $k-1$. Верни матрицу $M$ формы $(k, k)$: $M_{ij}$ — сколько объектов класса $i$ предсказаны как класс $j$. $n \le 10^6$, $k \le 300$.

```
confusion([0, 1, 1, 2], [0, 2, 1, 2], 3)
  → [[1, 0, 0], [0, 1, 1], [0, 0, 1]]

confusion([1, 1, 0], [0, 0, 0], 2)
  → [[1, 0], [2, 0]]
```